

P1690R1

Refinement Proposal for P0919

Heterogeneous lookup for unordered containers

Published Proposal, 2019-08-12

This version:

<http://wg21.link/p1690r1>

Authors:

[Xiao Shi \(Facebook\)](#) xshi.fb@gmail.com

[Mateusz Pusz \(Epm Systems\)](#) mateusz.pusz@gmail.com

[Geoffrey Romer \(Google\)](#) gromer@google.com

Audience:

LWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

This proposal outlines design and practical concerns with the mechanism to enable heterogeneous lookup in unordered associative containers in [\[P0919R3\]](#). We propose an alternative similar to the mechanism initially proposed in [\[P0919R0\]](#).

Table of Contents

- 1 Motivation and Scope
- 2 Design Concerns
 - 2.1 Consistency with Prior Art and Existing Practices
 - 2.2 Compatibility of [Hash](#) and [Pred](#)

2.3 Minimizing User Confusion

3 Impact On The Standard

4 Proposed Wording

5 Possible Future Extensions

6 Implementation Experiences

7 Acknowledgements

8 Revision History

8.1 r0 -> r1

References

Normative References

Informative References

§ 1. Motivation and Scope

Heterogeneous lookup allows unordered associative containers (`std::unordered_map`, `std::unordered_set`, etc.) to perform lookup operations on different (but compatible) types without creating a temporary key object. It is an important and useful feature, particularly for performance reasons. [P0919R0] initially proposed to enable heterogeneous look up when both the hash function `Hash` and the key equality comparison function `Pred` are tagged `is_transparent`. The current revision R3 revised to enable heterogeneous lookup when the hash function has a nested tag type `transparent_key_equal` which is tagged `is_transparent`. If heterogeneous lookup is enabled, the `Pred` template parameter is entirely ignored, and `Hash::transparent_key_equal` is used instead. The use of `Hash::transparent_key_equal`, however, deviates from prior art (the `is_transparent` tagging mechanism for ordered associative containers), does not address the incompatibility concerns originally expressed by LEWG members, and adds more subtle and confusing corner cases and will likely surprise and confuse the user.

§ 2. Design Concerns

§ 2.1. Consistency with Prior Art and Existing Practices

[N3657] introduced heterogeneous lookup support for ordered associative containers (`std::map`, `std::set`, etc) to the C++ Standard Library, which uses the `Compare::is_transparent` tag to enable. [P0919R3] deviates from this mechanism: `Hash::transparent_key_equal` must denote another type that specifies the `is_transparent` tag. It also causes the `Pred` template parameter to be ignored (even if it is explicitly specified), which we expect will be unintuitive to the users.

[P0919R3] does not standardize existing practice. [SwissTables] and [F14] Hash Tables, two of the most widely used C++ hash table implementations in Google and Facebook, both use the conjunction of `is_transparent` tags: heterogeneous lookup is only enabled when both `Hash::is_transparent` and `Pred::is_transparent` denote a type. See the Implementation Experiences Section for more details.

§ 2.2. Compatibility of `Hash` and `Pred`

LEWG raised a valid concern about the compatibility of the `Hash` and `Pred` during the review of [P0919R0]. Specifically, in the R0 mechanism, should the user neglects to tag one of the two functions as `is_transparent`, neglects to specify one of them in the template parameter, or if `Hash` and `Pred` operate on different sets of types, heterogeneous lookup would not be enabled.

[P0919R3] tries to address the compatibility concern by stipulating compilation failures for containers with incompatible `Hash` and `Pred` types. However, using `transparent_key_equal` does not preclude the possibility of incompatibility--the user could still specify an incompatible key equality comparator as `transparent_key_equal`. In this case, the heterogeneous lookup methods would be unavailable (via SFINAE) or fall back to creating a temporary `key_type` object, which exhibits exactly the same behavior as R0.

We argue that the compatibility concern can be addressed by the implementation via good defaults, compiler warnings, and existing compilation rules. In our experience, this has not posed a problem.

§ 2.3. Minimizing User Confusion

With [P0919R3], the user could specify the key equality comparator in two places: as the `Pred` template parameter or as `Hash::transparent_key_equal`. R3 tries to prevent misuse by stipulating the container will fail to compile if the `Pred` template parameter is not the same as

`transparent_key_equal` or (the default) `std::equal_to<Key>`. The problem is that users are likely confused or surprised when explicitly-supplied `Pred` template parameter is ignored.

EXAMPLE 1

In the following example, the program will compile but the container will be associated with two different `Pred` types `MyEqualTo` and `std::equal_to`, likely causing confusion for the user.

```
struct MyEqualTo {
    using is_transparent = ...;
    ...
};
struct MyHash {
    using is_transparent = ...;
    using transparent_key_equal = MyEqualTo;
    ...
};
// The user explicitly uses std::equal_to<K> as the 4th template parameter
// in fact that type is NEVER used:
std::unordered_map<K, V, MyHash, std::equal_to<K>> m;
```

In addition, allowing the `Hash` and `Pred` to operate on different sets of types can be useful. It is conceivable and useful for a hashing library to have a hasher that could operate on a wide variety of key types, e.g., `absl::Hash<T>` for string-like types or pointer types, `folly::Hash` for most common types. `Pred`, on the other hand, may not care about the entirety of the set of types the hasher could hash.

EXAMPLE 2

The user is unable to reuse the `CommonHasher` (e.g., in a common hashing library) with a different custom `Pred` comparator, without either modifying the common library (which may not be an option) or introducing yet another wrapper.

```
struct CommonEqualTo {...};
struct CommonHasher {
    using is_transparent = void;
    using transparent_key_equal = CommonEqualTo;
    ...
};
std::unordered_set<K, CommonHasher> s1; // works just fine

struct MyEqualTo {...};
std::unordered_set<K, CommonHasher, MyEqualTo> s2; // fails to compile
```

§ 3. Impact On The Standard

This proposal modifies the unordered associative containers in `<unordered_map>` and `<unordered_set>` by overloading the lookup member functions with member function templates.

There are no language changes.

Almost all existing C++17 code is unaffected because new member functions are disabled from overload resolution process unless both the `Hash` template parameter and the `Pred` template parameters have `is_transparent` property.

§ 4. Proposed Wording

The proposed changes are relative to the working draft of the standard as of [\[N4810\]](#).

Modify **22.2.7 [unord.req]** paragraph 11.7 as follows:

(11.7) `a_tran` denotes a possibly `const` value of type `X` when the ~~the qualified-id~~ `X::hasher::transparent_key_equal` ~~is valid and denotes a type (13.9.2);~~ qualified-ids `X::key_equal::is_transparent` and `X::hasher::is_transparent` are both valid and denote types (13.9.2).

Modify table 70 in section 22.2.7 [unord.req] as follows:

Expression	Return type	Assertion/note pre-/post-condition	Complexity
<code>X::key_equal</code>	<code>Hash::transparent_key_equal</code> if such a <i>qualified-id</i> is valid and denotes a type (13.9.2); otherwise, <code>Pred::Pred</code>	Requires: <code>key_equal</code> is <code>CopyConstructible</code> . <code>key_equal</code> shall be a binary predicate that takes two arguments of type <code>Key</code> . <code>key_equal</code> is an equivalence relation.	compile time

Modify paragraph 17 in 22.2.7 [unord.req]:

~~If the *qualified-id* `Hash::transparent_key_equal` is valid and denotes a type (12.9.2), then the program is ill-formed if either:~~

- ~~• *qualified-id* `Hash::transparent_key_equal::is_transparent` is not valid or does not denote a type, or~~
- ~~• `Pred` is a different type than `equal_to<Key>` or `Hash::transparent_key_equal`:~~

The member function templates `find`, `count`, `equal_range`, and `contains` shall not participate in overload resolution unless the ~~*qualified-id* `Hash::transparent_key_equal` is valid and denotes a type~~ *qualified-ids* `Pred::is_transparent` and `Hash::is_transparent` are both valid and denote types (13.9.2).

Modify paragraph 3 of 22.5.4.1 [unord.map.overview] as follows:

```

namespace std {
    template<class Key,
            class T,
            class Hash = hash<Key>,
            class Pred = equal_to<Key>,
            class Allocator = allocator<pair<const Key, T>>>
    class unordered_map {
    public:
        // types
        using key_type          = Key;
        using mapped_type       = T;
        using value_type        = pair<const Key, T>;
        using hasher             = Hash;
using key_equal               = see 22.2.7;
        using key_equal         = Pred;
        using allocator_type     = Allocator;
    };
};

```

Modify paragraph 3 of **22.5.5.1 [unord.multimap.overview]** as follows:

```

namespace std {
    template<class Key,
            class T,
            class Hash = hash<Key>,
            class Pred = equal_to<Key>,
            class Allocator = allocator<pair<const Key, T>>>
    class unordered_multimap {
    public:
        // types
        using key_type          = Key;
        using mapped_type       = T;
        using value_type        = pair<const Key, T>;
        using hasher             = Hash;
using key_equal               = see 22.2.7;
        using key_equal         = Pred;
        using allocator_type     = Allocator;
    };
};

```

Modify paragraph 3 of **22.5.6.1 [unord.set.overview]** add:

```

namespace std {
    template<class Key,
            class Hash = hash<Key>,
            class Pred = equal_to<Key>,
            class Allocator = allocator<pair<const Key, T>>>
    class unordered_set {
    public:
        // types
        using key_type      = Key;
        using value_type    = Key;
        using hasher        = Hash;
using key_equal          = see 22.2.7;
        using key_equal     = Pred;
        using allocator_type = Allocator;
    };
}

```

Modify paragraph 3 of **22.5.7.1 [unord.multiset.overview]** add:

```

namespace std {
    template<class Key,
            class Hash = hash<Key>,
            class Pred = equal_to<Key>,
            class Allocator = allocator<pair<const Key, T>>>
    class unordered_multiset {
    public:
        // types
        using key_type      = Key;
        using value_type    = Key;
        using hasher        = Hash;
using key_equal          = see 22.2.7;
        using key_equal     = Pred;
        using allocator_type = Allocator;
    };
}

```

§ 5. Possible Future Extensions

This mechanism can be extended to `operator[]` and other heterogeneous mutation methods.

§ 6. Implementation Experiences

Two widely used hash table implementations [\[SwissTables\]](#) and [\[F14\]](#) both enable heterogeneous operations when both `Hash::is_transparent` and `Pred::is_transparent` exists and denote a type. If either is not present or either does not take a specific type, heterogeneous operations won't be enabled for that type. The user may see their code fail to compile, or will not get the performance benefits (if the type implicitly creates a temporary `key_type` object). Either is a signal to double check the `Hash` and `Pred`. The vast majority of our users who elect to use heterogeneous operations did not run into any issue.

§ 7. Acknowledgements

We would like to thank Samuel Benzaquen, Nathan Bronson, Jay Feldblum, David Goldblatt, Chris Kennelly, Matthew Fowles Kulukundis, and Titus Winters for various discussions, insights, and implementation experiences.

§ 8. Revision History

§ 8.1. r0 -> r1

The paper (r0) was reviewed by LEWG at the 2019 Cologne meeting with the decision of forwarding to LWG for C++20.

	SF		F		N		A		SA	
	5		6		7		0		0	

R1 adds the [§8 Revision History](#) section and includes a typo fix in the first paragraph of [§4 Proposed Wording](#).

§ References

§ Normative References

[N3657]

J. Wakely, S. Lavavej, J. Muñoz. [Adding heterogeneous comparison lookup to associative containers \(rev 4\)](#). 19 March 2013. URL: <https://wg21.link/n3657>

[N4810]

Richard Smith. [Working Draft, Standard for Programming Language C++](https://wg21.link/n4810). 15 March 2019. URL: <https://wg21.link/n4810>

[P0919R0]

Mateusz Pusz. [Heterogeneous lookup for unordered containers](https://wg21.link/p0919r0). 8 February 2018. URL: <https://wg21.link/p0919r0>

[P0919R3]

Mateusz Pusz. [Heterogeneous lookup for unordered containers](https://wg21.link/p0919r3). 9 November 2018. URL: <https://wg21.link/p0919r3>

§ Informative References

[F14]

Nathan Bronson; Xiao Shi. [Open-sourcing F14 for faster, more memory-efficient hash tables](https://code.fb.com/developer-tools/f14/). URL: <https://code.fb.com/developer-tools/f14/>

[SwissTables]

Sam Benzaquen; et al. [Swiss Tables and absl::Hash](https://abseil.io/blog/20180927-swisstables). URL: <https://abseil.io/blog/20180927-swisstables>